NetBox 入門

aoki-taquan

2026 年 03 月 16 日

目次

1 はじめに 7
1.1 NetBox とは 7
1.2 なぜ NetBox を使うのか 7
1.2.1 ネットワーク管理の課題 7
1.2.2 NetBox が解決すること 7
1.3 このガイドについて 8
1.4 対象読者 8
1.5 前提知識 8
1.6 バージョン情報 8
2 インストール 9
2.1 システム要件 9
2.1.1 Docker を使ったインストール（推奨） 9
2.1.2 ベアメタル・仮想マシンへのインストール 9
2.2 Docker のインストール 9
2.2.1 Ubuntu / Debian 9
2.3 netbox-docker を使ったインストール 10
2.3.1 リポジトリのクローン 10
2.3.2 docker-compose.override.yml の作成 10
2.3.3 起動 10
2.3.4 起動確認 10
2.4 初期設定 11
2.4.1 Web UI へのアクセス 11
2.4.2 スーパーユーザーの作成 11
2.4.3 タイムゾーンの設定 11
2.5 設定ファイルのカスタマイズ 11
2.5.1 configuration.py の主要な設定 11
2.6 アップグレード 12
2.7 バックアップ 12
2.7.1 データベースのバックアップ 12
2.7.2 リストア 12
3 基本操作 13
3.1 Web UI の概要 13
3.1.1 トップナビゲーション 13
3.2 ダッシュボード 13
3.2.1 ウィジェットの追加 13
3.3 基本的なデータ操作 14
3.3.1 オブジェクトの作成 14
3.3.2 一括インポート 14
3.3.3 一括編集 14

3.4 検索機能 14
3.4.1 グローバル検索 14
3.4.2 フィルタリング 14
3.5 タグ 14
3.5.1 タグの作成 15
3.5.2 タグの適用 15
3.6 変更履歴（Change Log） 15
3.7 ユーザー管理 15
3.7.1 ユーザーの作成 15
3.7.2 パーミッション設定 15
3.8 ジャーナル（Journal） 15
4 DCIM（データセンターインフラ管理） 17
4.1 管理階層の概要 17
4.2 Region（地域） 17
4.2.1 作成例 17
4.3 Site（サイト） 17
4.3.1 主要フィールド 17
4.4 Location（ロケーション） 18
4.5 Rack（ラック） 18
4.5.1 主要フィールド 18
4.5.2 ラックビュー 18
4.6 Device Type（デバイスタイプ） 19
4.6.1 メーカー（Manufacturer）の登録 19
4.6.2 デバイスタイプの登録 19
4.6.3 コンポーネントテンプレート 19
4.6.4 コミュニティライブラリの活用 19
4.7 Device Role（デバイスロール） 19
4.8 Device（デバイス） 20
4.8.1 主要フィールド 20
4.8.2 インターフェースの管理 20
4.8.3 ケーブルの接続 20
4.9 Platform（プラットフォーム） 21
5 IPAM（IP アドレス管理） 22
5.1 IPAM の構造 22
5.2 RIR（Regional Internet Registry） 22
5.3 Aggregate（集約プレフィックス） 22
5.4 VRF（Virtual Routing and Forwarding） 22
5.4.1 VRF の作成 22
5.5 Prefix（プレフィックス） 23
5.5.1 主要フィールド 23
5.5.2 ステータスの意味 23
5.5.3 プレフィックスの可視化 23

5.5.4 利用可能な IP の検索 24
5.6 VLAN 24
5.6.1 VLAN グループ 24
5.6.2 VLAN の管理 24
5.7 IP Address（IP アドレス） 24
5.7.1 主要フィールド 24
5.7.2 インターフェースへの割り当て 25
5.8 IP Range（IP 範囲） 25
5.9 IPAM のベストプラクティス 25
5.9.1 プレフィックスの役割定義 25
5.9.2 階層的なプレフィックス設計例 26
6 仮想化管理 27
6.1 管理構造 27
6.2 Cluster Type（クラスタータイプ） 27
6.3 Cluster（クラスター） 27
6.3.1 主要フィールド 27
6.3.2 クラスターへのホスト割り当て 28
6.4 Virtual Machine（仮想マシン） 28
6.4.1 主要フィールド 28
6.4.2 VM インターフェースの管理 28
6.4.3 IP アドレスの割り当て 28
6.5 テナント（Tenant）管理 28
6.5.1 テナントグループの作成 29
6.5.2 テナントの割り当て 29
6.6 仮想化管理の活用例 29
6.6.1 クラウド環境との併用 29
6.6.2 Ansible / Terraform との連携 29
7 回路と接続管理 30
7.1 ケーブル管理 30
7.1.1 ケーブルの作成 30
7.1.2 ケーブルのフィールド 30
7.1.3 接続可能なオブジェクト 30
7.1.4 パッチパネルの管理 30
7.2 Wireless（ワイヤレス LAN） 30
7.2.1 Wireless LAN Group 31
7.2.2 Wireless LAN 31
7.3 回路（Circuits）管理 31
7.3.1 Provider（プロバイダー） 31
7.3.2 Provider Network 31
7.3.3 Circuit Type（回線タイプ） 31
7.3.4 Circuit（回線） 31
7.3.5 Circuit Termination（回線終端） 32

7.4 電源管理 32
7.4.1 Power Panel（電源パネル） 32
7.4.2 Power Feed（電源フィード） 32
8 カスタマイズ 34
8.1 カスタムフィールド（Custom Fields） 34
8.1.1 カスタムフィールドの作成 34
8.1.2 フィールドタイプ 34
8.1.3 設定項目 34
8.1.4 活用例 35
8.2 カスタム選択肢（Custom Field Choices） 35
8.3 カスタムリンク（Custom Links） 35
8.3.1 活用例 36
8.4 エクスポートテンプレート（Export Templates） 36
8.4.1 活用例（Ansible インベントリの生成） 36
8.5 Webhook 36
8.5.1 主要設定 36
8.5.2 活用例 37
8.6 スクリプト（Scripts） 37
8.6.1 スクリプトの例 37
8.7 設定コンテキスト（Config Contexts） 38
8.7.1 活用例 38
9 API と自動化 39
9.1 API トークンの発行 39
9.1.1 Web UI からの発行 39
9.1.2 API トークンの管理 39
9.2 REST API の基本 39
9.2.1 API ドキュメント 39
9.2.2 基本的なリクエスト 39
9.2.3 フィルタリング 40
9.2.4 ページネーション 40
9.3 Python からの API 利用 40
9.3.1 requests ライブラリを使う場合 40
9.3.2 pynetbox ライブラリ（推奨） 41
9.4 GraphQL API 42
9.5 Ansible との連携 42
9.5.1 netbox.netbox コレクション 42
9.5.2 Dynamic Inventory（動的インベントリ） 42
9.5.3 Ansible モジュールの使用例 43
9.6 Terraform との連携 43
9.7 NAPALM 連携 44
10 運用管理 45
10.1 バックアップとリストア 45

10.1.1 データベースのバックアップ 45
10.1.2 リストア 45
10.2 アップグレード 46
10.2.1 アップグレード前の確認 46
10.2.2 アップグレード手順（netbox-docker） 46
10.3 パフォーマンスチューニング 46
10.3.1 データベースの最適化 46
10.3.2 Redis キャッシュ 46
10.3.3 Workers の設定 47
10.4 監視 47
10.4.1 ヘルスチェックエンドポイント 47
10.4.2 アクセスログ 47
10.4.3 Prometheus / Grafana との連携 47
10.5 プラグイン 47
10.5.1 主要なプラグイン 48
10.5.2 プラグインのインストール（Docker） 48
10.6 セキュリティのベストプラクティス 48
10.6.1 本番環境向けの設定 48
10.6.2 LDAP / SSO 連携 49
10.6.3 定期的なセキュリティタスク 49
10.7 よくあるトラブルと対処法 50
10.7.1 DB 接続エラー 50
10.7.2 マイグレーションの失敗 50
10.7.3 パフォーマンス低下 50

1 はじめに

1.1 NetBox とは

NetBox は、ネットワークインフラ管理のためのオープンソースプラットフォームです。DigitalOcean 社が開発し、現在は NetBox Labs が主導するコミュニティによって維持されています。ネットワーク機器やケーブル、IP アドレスなどの情報を一元管理する「ネットワークの信頼できる唯一の情報源（Source of Truth）」として機能します。

主な機能は以下の通りです：

- **DCIM（Data Center Infrastructure Management）**：データセンターの物理インフラ管理
- **IPAM（IP Address Management）**：IP アドレス・プレフィックス・VLAN の管理
- **仮想化管理**：仮想マシンやクラスターの管理
- **回路管理**：WAN 回線や接続情報の管理
- **REST API**：外部システムとの連携や自動化
- **カスタマイズ**：カスタムフィールド・タグ・Webhook による拡張

1.2 なぜ NetBox を使うのか

1.2.1 ネットワーク管理の課題

多くの組織では、ネットワーク情報を以下の方法で管理しています：

- Excel / スプレッドシート
- 独自のスクリプト
- 複数のバラバラなツール

これらの方法では、データの分散・重複・陳腐化が生じやすく、障害対応や構成変更時に正確な情報を素早く得ることが困難です。

1.2.2 NetBox が解決すること

課題	従来の方法	NetBox
IP アドレス管理	Excel で管理、重複・漏れが発生	階層的なプレフィックス管理で重複を防止
機器情報	個別ドキュメントに分散	統一されたデバイス DB
ラック構成	図面や写真	インタラクティブなラックビュー
変更追跡	手動更新・バージョン管理なし	変更履歴（Change Log）を自動記録
自動化連携	困難	REST API / GraphQL で容易

1.3 このガイドについて

本書は NetBox を初めて使う方を対象に、インストールから基本操作、DCIM・IPAM の設定、API を使った自動化まで網羅的に解説する入門書です。

1.4 対象読者

- ネットワーク管理者・インフラエンジニア
- IP アドレス管理を整備したい方
- ネットワーク機器の台帳管理を自動化したい方
- Python や API を使ったネットワーク自動化に興味がある方

1.5 前提知識

- 基本的な Linux コマンド操作
- ネットワークの基礎知識（IP アドレス、サブネット、VLAN、BGP など）
- Docker の基本操作（インストール方法に Docker を使用）
- HTTP / REST API の基本概念があると望ましい

1.6 バージョン情報

本書は NetBox v4.x をベースに執筆しています。バージョンによって UI や機能が異なる場合がありますが、基本的な概念は共通です。

2 インストール

2.1 システム要件

NetBox を動作させるための要件は以下の通りです。

2.1.1 Docker を使ったインストール（推奨）

本書では Docker Compose を使ったインストール方法を解説します。 最もシンプルで再現性が高い方法です。

- **OS**：Linux（Ubuntu 22.04 / Debian 12 推奨）、macOS、Windows（WSL2）
- **CPU**：2 コア以上
- **メモリ**：4 GB 以上（推奨 8 GB）
- **ストレージ**：20 GB 以上
- **Docker**：24.0 以上
- **Docker Compose**：2.0 以上

2.1.2 ベアメタル・仮想マシンへのインストール

Python 3.10 以上、PostgreSQL 14 以上、Redis 7.0 以上が必要です。 本書では Docker Compose 方式を推奨します。

2.2 Docker のインストール

2.2.1 Ubuntu / Debian

```
# 古いバージョンのアンインストール
sudo apt remove docker docker-engine docker.io containerd runc

# 必要パッケージのインストール
sudo apt update
sudo apt install -y ca-certificates curl gnupg

# Docker の公式 GPG キーを追加
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# リポジトリの追加
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
# Docker のインストール
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin

# 現在のユーザーを docker グループに追加（再ログイン後に有効）
sudo usermod -aG docker $USER
```

2.3 netbox-docker を使ったインストール

NetBox 公式が提供する `netbox-docker` を使うと、簡単に Docker 環境を構築できます。

2.3.1 リポジトリのクローン

```
git clone -b release https://github.com/netbox-community/netbox-docker.git
cd netbox-docker
```

2.3.2 docker-compose.override.yml の作成

```
tee docker-compose.override.yml <<EOF
services:
  netbox:
    ports:
      - 8000:8080
EOF
```

2.3.3 起動

```
docker compose pull
docker compose up -d
```

初回起動時はイメージのダウンロードと DB の初期化が行われます。完了まで数分かかる場合があります。

2.3.4 起動確認

```
docker compose ps
```

すべてのサービスが `running` または `healthy` になっていれば成功です。

```
NAME                    STATUS
netbox-docker-netbox    running (healthy)
```

```
netbox-docker-postgres  running (healthy)
netbox-docker-redis    running (healthy)
```

2.4 初期設定

2.4.1 Web UI へのアクセス

ブラウザで以下の URL にアクセスします：

```
http://<サーバーのIPアドレス>:8000
```

デフォルトの管理者アカウントは以下の通りです：

- **ユーザー名**：`admin`
- **パスワード**：`admin`

ログイン後、速やかにパスワードを変更してください。

2.4.2 スーパーユーザーの作成

```
docker compose exec netbox /opt/netbox/netbox/manage.py createsuperuser
```

2.4.3 タイムゾーンの設定

`docker-compose.override.yml` に環境変数を追加します：

```yaml
services:
  netbox:
    environment:
      - TIME_ZONE=Asia/Tokyo
```

変更後に再起動します：

```
docker compose up -d
```

2.5 設定ファイルのカスタマイズ

2.5.1 configuration.py の主要な設定

`env/netbox.env` または `configuration/configuration.py` で主要な設定を変更できます。

```
# 未認証アクセスを禁止（ログインを必須化）
LOGIN_REQUIRED = True
```

```
# セッションのタイムアウト（秒）
LOGIN_TIMEOUT = 1800

# ページあたりの表示件数
PAGINATE_COUNT = 50

# 許可するホスト名
ALLOWED_HOSTS = ['netbox.example.com', '192.168.1.10']
```

2.6 アップグレード

```
cd netbox-docker
git pull
docker compose pull
docker compose up -d
```

2.7 バックアップ

2.7.1 データベースのバックアップ

```
docker compose exec postgres pg_dump -U netbox netbox > netbox_backup.sql
```

2.7.2 リストア

```
docker compose exec -T postgres psql -U netbox netbox < netbox_backup.sql
```

3 基本操作

3.1 Web UI の概要

NetBox の Web UI はシンプルなナビゲーション構造を持っています。

3.1.1 トップナビゲーション

画面上部のナビゲーションバーには以下のメニューがあります：

メニュー	内容
Organization	サイト・テナント・ロケーション・連絡先の管理
Devices	デバイス・ラック・コンソール・電源の管理
Connections	ケーブル・インターフェース接続・回線の管理
Wireless	ワイヤレス LAN の管理
IPAM	IP アドレス・プレフィックス・VLAN・VRF の管理
VPN	トンネル・L2VPN の管理
Virtualization	仮想マシン・クラスターの管理
Circuits	WAN 回線・プロバイダーの管理
Power	電源パネル・電源フィードの管理
Customization	カスタムフィールド・タグ・スクリプトの管理
Operations	変更ログ・ジャーナル・バックグラウンドジョブの管理

3.2 ダッシュボード

ログイン後に表示されるダッシュボードは、よく使う情報を一覧表示するカスタマイズ可能なページです。

3.2.1 ウィジェットの追加

右上の「+ Add Widget」ボタンから以下のウィジェットを追加できます：

- IP アドレスの使用状況
- 最近の変更履歴
- 任意のオブジェクト一覧

3.3 基本的なデータ操作

3.3.1 オブジェクトの作成

各管理画面の右上にある「+ Add」ボタンをクリックしてオブジェクトを作成します。

3.3.2 一括インポート

CSV ファイルを使って複数のオブジェクトを一括登録できます。

1. 対象の一覧画面を開く（例：Devices）
2. 右上の「Import」ボタンをクリック
3. CSV ヘッダーとデータを入力または貼り付け
4. 「Submit」で一括登録

CSV フォーマットの例（デバイスの場合）：

```
name,device_role,device_type,site,status
sw-01,switch,Cisco Catalyst 9300,Tokyo-DC,active
sw-02,switch,Cisco Catalyst 9300,Tokyo-DC,active
```

3.3.3 一括編集

一覧画面でチェックボックスを選択し、「Edit Selected」で複数オブジェクトを一括編集できます。

3.4 検索機能

3.4.1 グローバル検索

画面右上の検索ボックス（または `/` キー）で、全オブジェクトを横断検索できます。

3.4.2 フィルタリング

各一覧画面には詳細なフィルタパネルがあります。 「Filter」ボタンをクリックして条件を指定します。

よく使うフィルタ例：

```
# デバイス一覧でサイトと状態でフィルタ
Site: Tokyo-DC
Status: Active
```

3.5 タグ

タグを使うことで、様々なオブジェクトを横断的に分類できます。

3.5.1 タグの作成

Customization > Tags から新しいタグを作成します。

- **Name**：タグ名（例：`production`）
- **Color**：表示色
- **Description**：説明

3.5.2 タグの適用

オブジェクトの編集フォームで「Tags」フィールドにタグを入力します。複数のタグを適用でき、後から一覧フィルタで絞り込みが可能です。

3.6 変更履歴（Change Log）

Operations > Change Log から、すべてのオブジェクトへの変更履歴を確認できます。

表示される情報：

- 変更日時
- 変更ユーザー
- 変更されたオブジェクト
- 変更の種類（作成・変更・削除）
- 変更前後の差分（JSON 形式）

3.7 ユーザー管理

3.7.1 ユーザーの作成

Admin パネル（`/admin/`）または API からユーザーを作成します。

3.7.2 パーミッション設定

NetBox では細かい権限設定が可能です：

権限レベル	内容
View	読み取りのみ
Add	オブジェクトの作成
Change	オブジェクトの編集
Delete	オブジェクトの削除

トークンベースの API 認証も同じユーザー権限に従います。

3.8 ジャーナル（Journal）

各オブジェクトには「ジャーナル」機能があり、作業メモや変更理由を記録できます。

オブジェクトの詳細画面下部の「Journal」タブからメモを追加します。マークダウン形式で記述できます。

4 DCIM（データセンターインフラ管理）

DCIM（Data Center Infrastructure Management）は、NetBox のコア機能の一つです。物理的なネットワークインフラを階層的に管理します。

4.1 管理階層の概要

NetBox の DCIM は以下の階層構造で管理します：

```
Region（地域）
  └── Site Group（サイトグループ）
        └── Site（サイト）
              └── Location（ロケーション）
                    └── Rack（ラック）
                          └── Device（デバイス）
                                └── Interface / Port
```

4.2 Region（地域）

Organization > Regions から地域を管理します。

地域は階層化できます（例：日本 > 東日本 > 東京）。

4.2.1 作成例

```
Name: Japan
Slug: japan
```

```
Name: Tokyo
Slug: tokyo
Parent: Japan
```

4.3 Site（サイト）

サイトはデータセンター、オフィスなどの物理的な拠点を表します。

4.3.1 主要フィールド

フィールド	説明
Name	サイト名（例：Tokyo-DC1）
Slug	URL 用の識別子（例：tokyo-dc1）
Status	Active / Planned / Retired など

フィールド	説明
Region	所属するリージョン
Facility	施設名・住所
ASN	BGP ASN 番号
Time zone	タイムゾーン
Description	説明

4.4 Location（ロケーション）

サイト内の部屋やフロアなどの区画を表します。

例：1F サーバールーム、2F 機器室

4.5 Rack（ラック）

物理的なラックキャビネットを管理します。

4.5.1 主要フィールド

フィールド	説明
Name	ラック名（例：Rack-01）
Site	設置サイト
Location	設置ロケーション（任意）
Status	Active / Planned など
Role	ラックの用途（例：Network, Server）
Type	ラックのタイプ（2-post, 4-post, Wall frame など）
Width	幅（19 インチ、23 インチ）
U Height	高さ（U 数）
Desc Units	ユニット番号の方向（上から/下から）

4.5.2 ラックビュー

ラックの詳細画面では、インタラクティブなラック図が表示されます。各 U に配置されたデバイスが視覚的に確認できます。

4.6 Device Type（デバイスタイプ）

デバイスタイプはメーカーとモデルの組み合わせを定義するテンプレートです。

4.6.1 メーカー（Manufacturer）の登録

Devices > Manufacturers から登録します。

例：Cisco、Juniper、Arista、HPE

4.6.2 デバイスタイプの登録

Devices > Device Types から登録します。

フィールド	説明
Manufacturer	メーカー（例：Cisco）
Model	モデル名（例：Catalyst 9300-48P）
Slug	識別子（例：catalyst-9300-48p）
U Height	搭載 U 数（例：1）
Is full depth	全奥行き占有か否か
Front Image	前面写真（任意）
Rear Image	背面写真（任意）

4.6.3 コンポーネントテンプレート

デバイスタイプにはインターフェースやポートのテンプレートを定義できます。 新しいデバイスを作成するとテンプレートが自動適用されます。

- **Interfaces**：GigabitEthernet0/0、TenGigabitEthernet1/0/1 など
- **Console Ports**：コンソールポート
- **Power Ports**：電源ポート
- **Module Bays**：モジュールベイ（スタック・ラインカード）

4.6.4 コミュニティライブラリの活用

`netbox-community/devicetype-library` リポジトリに多数のデバイスタイプが登録されています。YAML 形式でインポートできます。

```
git clone https://github.com/netbox-community/devicetype-library.git
```

4.7 Device Role（デバイスロール）

デバイスロールは機器の役割を定義します。

例：Router、Switch、Firewall、Server、PDU、Patch Panel

Devices > Device Roles から作成します。

4.8 Device（デバイス）

実際のネットワーク機器を登録します。

4.8.1 主要フィールド

フィールド	説明
Name	ホスト名（例：core-sw-01）
Device Role	役割（例：Core Switch）
Device Type	機種（例：Cisco Catalyst 9300）
Site	設置サイト
Rack	設置ラック
Position	ラック内の U 位置
Face	前面/背面
Status	Active / Staged / Offline など
Platform	OS プラットフォーム（例：Cisco IOS-XE）
Serial Number	シリアル番号
Asset Tag	資産タグ
Primary IPv4	主要 IPv4 アドレス
Primary IPv6	主要 IPv6 アドレス

4.8.2 インターフェースの管理

デバイス詳細画面の「Interfaces」タブから各インターフェースを管理します。

インターフェースの種別：

- Physical（物理）：1000BASE-T、10GBASE-SR など
- Virtual（仮想）：VLAN サブインターフェース、Loopback など
- LAG（Link Aggregation）：ポートチャネル、LACP

4.8.3 ケーブルの接続

インターフェースの詳細画面から「Connect」ボタンで対向機器のポートと接続を登録します。 接続情報はケーブルとして記録されます。

4.9 Platform（プラットフォーム）

デバイスの OS・プラットフォームを定義します。

例：Cisco IOS-XE、Juniper Junos、Arista EOS、Linux

NAPALM や Nornir などの自動化ツールと連携する場合に使用します。

5 IPAM（IP アドレス管理）

IPAM（IP Address Management）は NetBox の中核機能です。 IP アドレス空間を階層的・可視的に管理します。

5.1 IPAM の構造

```
RIR（地域インターネットレジストリ）
  └── Aggregate（割り当てブロック）
        └── Prefix（プレフィックス）
              └── IP Range（IP 範囲）
              └── IP Address（個別 IP）
```

5.2 RIR（Regional Internet Registry）

IPAM > RIRs からインターネットレジストリを登録します。

代表的な RIR：

- APNIC（アジア太平洋）
- ARIN（北米）
- RIPE NCC（欧州・中東）
- LACNIC（中南米）
- AFRINIC（アフリカ）
- RFC 1918（プライベートアドレス）

5.3 Aggregate（集約プレフィックス）

自組織に割り当てられた最上位のアドレスブロックを登録します。

例：

- `10.0.0.0/8`（プライベート）
- `192.168.0.0/16`（プライベート）
- `203.0.113.0/24`（インターネットルーティング可能）

5.4 VRF（Virtual Routing and Forwarding）

VRF を使うことで、同じ IP アドレス空間を複数テナントや用途で分離管理できます。

5.4.1 VRF の作成

IPAM > VRFs から作成します。

フィールド	説明
Name	VRF 名（例：Production、Management）
RD	Route Distinguisher（例：65001:100）
Enforce Unique	重複 IP の登録を禁止するか
Description	説明

VRF の「Global」は VRF なし（グローバルルーティングテーブル）を意味します。

5.5 Prefix（プレフィックス）

プレフィックスはサブネットやアドレスブロックを管理します。

5.5.1 主要フィールド

フィールド	説明
Prefix	CIDR 表記（例：192.168.1.0/24）
VRF	所属 VRF
Status	Active / Reserved / Deprecated / Container
Role	用途（例：Infrastructure、Loopback、Management）
Site	関連するサイト
VLAN	関連する VLAN
Is a pool	IP プールとして扱う（DHCP 範囲など）
Mark Utilized	すべての IP が使用済みとみなすか
Description	説明

5.5.2 ステータスの意味

- **Active**：現在使用中
- **Reserved**：予約済み（将来の使用のために確保）
- **Deprecated**：廃止予定
- **Container**：子プレフィックスを含む集約コンテナ

5.5.3 プレフィックスの可視化

プレフィックス一覧では使用状況がプログレスバーで表示されます。 詳細画面では子プレフィックスと IP アドレスの一覧が確認できます。

5.5.4 利用可能な IP の検索

プレフィックス詳細画面の「Available IPs」タブで、 未割り当ての IP アドレスを一覧表示し、すぐに割り当てることができます。

5.6 VLAN

5.6.1 VLAN グループ

サイトやロケーション単位で VLAN グループを作成し、その中で VLAN を管理します。

IPAM > VLAN Groups から作成します。

5.6.2 VLAN の管理

IPAM > VLANs から VLAN を作成します。

フィールド	説明
VID	VLAN ID（1〜4094）
Name	VLAN 名（例：Management、Production）
Site	所属サイト
Group	VLAN グループ
Status	Active / Reserved / Deprecated
Role	用途（例：User Access、Transit）
Tenant	所属テナント（任意）

5.7 IP Address（IP アドレス）

個々の IP アドレスを管理します。

5.7.1 主要フィールド

フィールド	説明
Address	IP/プレフィックス長（例：192.168.1.1/24）
VRF	所属 VRF
Status	Active / Reserved / DHCP / SLAAC など
Role	Loopback / Secondary / Anycast / VIP など
DNS Name	対応する DNS 名
Assigned Object	割り当て先（インターフェース）

フィールド	説明
NAT（Inside）	NAT 変換前のアドレス
Description	説明

5.7.2 インターフェースへの割り当て

IP アドレスを機器のインターフェースに割り当てることで、どの機器のどのインターフェースがその IP を持っているかを管理できます。

```
Assigned Object: core-sw-01 > GigabitEthernet0/0
```

割り当てた IP は、デバイスの「Primary IP」として設定することもできます。

5.8 IP Range（IP 範囲）

連続した IP アドレスの範囲を管理します。DHCP スコープや特定用途の IP 範囲の管理に便利です。

フィールド	説明
Start Address	開始 IP（例：192.168.1.100/24）
End Address	終了 IP（例：192.168.1.200/24）
VRF	所属 VRF
Status	Active / Reserved など
Role	用途（例：DHCP Pool）
Size	範囲内の IP 数（自動計算）

5.9 IPAM のベストプラクティス

5.9.1 プレフィックスの役割定義

用途別のロールを作成し、プレフィックスに割り当てます：

- `Infrastructure`：ネットワーク機器間の接続
- `Loopback`：ルーターの Loopback アドレス
- `Management`：管理用アドレス帯
- `Server`：サーバーセグメント
- `Client`：クライアントセグメント
- `Transit`：WAN 回線の接続先

5.9.2 階層的なプレフィックス設計例

```
10.0.0.0/8          [Container] 全プライベート
  10.0.0.0/16       [Container] 東京DC
    10.0.1.0/24     [Active]    東京DC サーバーセグメント
    10.0.2.0/24     [Active]    東京DC 管理セグメント
  10.1.0.0/16       [Container] 大阪DC
    10.1.1.0/24     [Active]    大阪DC サーバーセグメント
```

6 仮想化管理

NetBox では仮想マシンやコンテナのインフラ情報も管理できます。

6.1 管理構造

```
Cluster Type（クラスタータイプ）
  └── Cluster（クラスター）
        └── Virtual Machine（仮想マシン）
              └── VM Interface（仮想インターフェース）
                    └── IP Address（IP アドレス）
```

6.2 Cluster Type（クラスタータイプ）

仮想化プラットフォームの種類を定義します。

Virtualization > Cluster Types から作成します。

例：

- VMware vSphere
- Proxmox VE
- OpenStack
- Kubernetes
- KVM / libvirt

6.3 Cluster（クラスター）

仮想化ホストのグループ（クラスター）を管理します。

6.3.1 主要フィールド

フィールド	説明
Name	クラスター名（例：vsphere-prod-01）
Type	クラスタータイプ（例：VMware vSphere）
Group	クラスターグループ（任意）
Site	設置サイト
Status	Active / Planned / Staged など
Description	説明

6.3.2 クラスターへのホスト割り当て

クラスター詳細画面の「Devices」タブから、 物理ホストサーバー（Device）をクラスターのメンバーとして割り当てます。

6.4 Virtual Machine（仮想マシン）

仮想マシンの情報を管理します。

6.4.1 主要フィールド

フィールド	説明
Name	VM 名（例：web-server-01）
Status	Active / Staged / Offline など
Cluster	所属クラスター
Device	稼働する物理ホスト（任意）
Role	役割（例：Web Server、DB Server）
Platform	OS プラットフォーム（例：Ubuntu 22.04）
Primary IPv4	主要 IPv4 アドレス
Primary IPv6	主要 IPv6 アドレス
vCPUs	仮想 CPU 数
Memory（MB）	メモリ量（MB）
Disk（GB）	ディスク容量（GB）
Tenant	所属テナント

6.4.2 VM インターフェースの管理

仮想マシンの詳細画面の「Interfaces」タブから仮想 NIC を管理します。

VLAN のタグ付け（Tagged / Untagged）も設定できます。

6.4.3 IP アドレスの割り当て

VM インターフェースに IP アドレスを割り当てます。 割り当て方法は物理デバイスと同様です。

6.5 テナント（Tenant）管理

テナント機能を使うことで、VM やネットワークリソースを 組織・部署・顧客単位で管理できます。

6.5.1 テナントグループの作成

Organization > Tenant Groups から作成します。

例：

```
Tenant Group: Business Units
  Tenant: Sales
  Tenant: Engineering
  Tenant: Operations
```

6.5.2 テナントの割り当て

多くのオブジェクト（VM、デバイス、プレフィックス、VLAN など）に テナントを割り当てることができます。

テナントフィルタを使うと、特定テナントのリソースのみを一覧表示できます。

6.6 仮想化管理の活用例

6.6.1 クラウド環境との併用

NetBox は物理環境だけでなく、クラウド環境の論理リソースも管理できます：

- AWS VPC → Prefix + VRF
- EC2 インスタンス → Virtual Machine
- Security Group → Tag + カスタムフィールド
- ELB → IP Address（VIP ロール）

6.6.2 Ansible / Terraform との連携

NetBox の API を使って、Ansible のインベントリソースや Terraform のデータソースとして活用できます。

```yaml
# Ansible dynamic inventory の例（netbox_inventory プラグイン）
# inventory/netbox.yml
plugin: netbox.netbox.nb_inventory
api_endpoint: http://netbox.example.com
token: your-api-token
group_by:
  - site
  - role
  - platform
```

7 回路と接続管理

7.1 ケーブル管理

NetBox では物理的なケーブル接続を管理できます。

7.1.1 ケーブルの作成

デバイスのインターフェース詳細画面から「Connect」ボタンで接続を作成します。

7.1.2 ケーブルのフィールド

フィールド	説明
Side A	接続元（デバイス・インターフェース）
Side B	接続先（デバイス・インターフェース）
Type	ケーブル種別（CAT6、CAT6A、Fiber SMF など）
Status	Connected / Planned / Decommissioning
Label	ケーブルラベル・識別番号
Color	ケーブルの色
Length	長さ（m / ft など）
Description	説明

7.1.3 接続可能なオブジェクト

ケーブルで接続できるのはインターフェースだけでなく、以下も対象です：

- Console Port / Console Server Port
- Power Port / Power Outlet
- Front Port / Rear Port（パッチパネル）

7.1.4 パッチパネルの管理

パッチパネルはデバイスタイプとして登録し、 Front Port と Rear Port をテンプレートで定義します。

ケーブルパスの追跡（Cable Trace）機能で、 パッチパネルを経由した接続経路を視覚的に確認できます。

7.2 Wireless（ワイヤレス LAN）

無線 LAN インフラを管理します。

7.2.1 Wireless LAN Group

ワイヤレス LAN グループを作成します。

例：`Office-WiFi`、`Guest-WiFi`

7.2.2 Wireless LAN

フィールド	説明
SSID	ネットワーク名
Group	ワイヤレス LAN グループ
Status	Active / Reserved / Disabled
Auth Type	認証方式（Open、WEP、WPA Personal など）
Auth Cipher	暗号化方式（AES、TKIP）
Auth PSK	事前共有鍵
VLAN	関連 VLAN

7.3 回路（Circuits）管理

WAN 回線や通信事業者との契約情報を管理します。

7.3.1 Provider（プロバイダー）

通信事業者・ISP を登録します。

Circuits > Providers から作成します。

例：NTT、KDDI、SoftBank、IIJ、AWS Direct Connect

7.3.2 Provider Network

プロバイダーが管理するネットワーク（クラウド側など）を登録します。

7.3.3 Circuit Type（回線タイプ）

回線の種類を定義します。

Circuits > Circuit Types から作成します。

例：Internet（インターネット回線）、MPLS、Dark Fiber、SD-WAN

7.3.4 Circuit（回線）

実際の WAN 回線を管理します。

フィールド	説明
CID	回線 ID・契約番号
Provider	通信事業者
Type	回線タイプ
Status	Active / Planned / Provisioning など
Tenant	利用テナント
Commit Rate（Kbps）	契約帯域
Description	説明

7.3.5 Circuit Termination（回線終端）

回線の接続先（自拠点のインターフェース）を登録します。

回線は Side A・Side Z の 2 つの終端を持ちます。

例：

```
Circuit: CID-001234
  Termination A: Tokyo-DC / core-rt-01 / GigabitEthernet0/0
  Termination Z: Provider Network（ISP 側）
```

7.4 電源管理

7.4.1 Power Panel（電源パネル）

データセンターの電源パネル（分電盤）を管理します。

Devices > Power Panels から作成します。

7.4.2 Power Feed（電源フィード）

電源パネルから各ラックへの電源供給を管理します。

フィールド	説明
Name	フィード名（例：Feed-A、Feed-B）
Power Panel	電源パネル
Rack	接続ラック
Status	Active / Planned など
Type	Primary / Redundant

フィールド	説明
Supply	AC / DC
Phase	Single-phase / Three-phase
Voltage	電圧（例：100、200）
Amperage	電流（A）

デバイスの Power Port を Power Feed に接続することで、 電源経路を追跡できます。

8 カスタマイズ

NetBox は柔軟なカスタマイズ機能を提供しています。

8.1 カスタムフィールド（Custom Fields）

標準フィールドにない情報を追加できます。

8.1.1 カスタムフィールドの作成

Customization > Custom Fields から作成します。

8.1.2 フィールドタイプ

タイプ	説明
Text	短いテキスト
Long text	長いテキスト（マークダウン対応）
Integer	整数
Decimal	小数
Boolean	真偽値（チェックボックス）
Date	日付
Date & time	日時
URL	URL
JSON	JSON データ
Selection	プルダウン選択（値は別途定義）
Multiple selection	複数選択
Object	別の NetBox オブジェクトへの参照
Multiple objects	複数オブジェクトへの参照

8.1.3 設定項目

設定	説明
Name	フィールド名（英数字・アンダースコア）
Label	UI 表示名
Content Types	適用するオブジェクト種別（複数可）

設定	説明
Required	必須入力にするか
Default	デフォルト値
Filter Logic	フィルタリングの挙動
UI Visibility	UI での表示方法
Weight	表示順（数値が大きいほど下位）
Group Name	グループ化の名前

8.1.4 活用例

```
カスタムフィールド名: lifecycle_end_date
ラベル: 保守期限
タイプ: Date
適用対象: Device, Virtual Machine
必須: No
```

```
カスタムフィールド名: monitoring_url
ラベル: 監視 URL
タイプ: URL
適用対象: Device, Virtual Machine
必須: No
```

8.2 カスタム選択肢（Custom Field Choices）

Selection タイプのカスタムフィールドで使用する選択肢を管理します。

Customization > Custom Field Choice Sets から作成します。

例：

```
Choice Set: 運用フェーズ
  choices:
    - planning（計画中）
    - testing（テスト中）
    - production（本番稼働）
    - eol（EOL）
```

8.3 カスタムリンク（Custom Links）

オブジェクト詳細画面に外部リンクや内部リンクを追加できます。

Customization > Custom Links から作成します。

8.3.1 活用例

監視システムへのリンク：

```
Name: Zabbix（監視）
Content Types: Device
URL: https://zabbix.example.com/zabbix/hosts.php?filter_host={{ object.name }}
```

ドキュメントへのリンク：

```
Name: Confluence ドキュメント
Content Types: Device
URL: https://wiki.example.com/search?q={{ object.name }}
```

8.4 エクスポートテンプレート（Export Templates）

オブジェクトの一覧を独自フォーマットでエクスポートできます。

Customization > Export Templates から Jinja2 テンプレートを作成します。

8.4.1 活用例（Ansible インベントリの生成）

```
[all]
{% for device in queryset %}
{{ device.name }} ansible_host={{ device.primary_ip4.address |
ansible.utils.ipaddr('address') }}
{% endfor %}

[{{ group_by }}]
{% for device in queryset %}
{{ device.name }}
{% endfor %}
```

8.5 Webhook

オブジェクトの作成・変更・削除時に外部システムへ HTTP リクエストを送信できます。

Customization > Webhooks から作成します。

8.5.1 主要設定

設定	説明
Name	Webhook 名
Content Types	対象オブジェクト種別
Events	作成・更新・削除のいずれかを選択

設定	説明
URL	送信先 URL
HTTP Method	POST / PUT / PATCH など
HTTP Content Type	application/json など
Additional Headers	追加 HTTP ヘッダー
Body Template	送信ボディの Jinja2 テンプレート
Secret	HMAC 署名用シークレット
SSL Verification	TLS 証明書の検証
Conditions	送信条件のフィルタ（JSON）

8.5.2 活用例

新しいデバイスが登録されたときに Slack へ通知：

```
URL: https://hooks.slack.com/services/XXX/YYY/ZZZ
Event: Created
Content Type: Device
Body Template:
{
  "text": "新しいデバイスが登録されました: {{ data.name }} @ {{ data.site.name }}"
}
```

8.6 スクリプト（Scripts）

カスタム Python スクリプトを Web UI から実行できます。

Customization > Scripts からスクリプトを管理します。

8.6.1 スクリプトの例

一括でデバイスのステータスを変更するスクリプト：

```python
from netbox.scripts import Script, MultiObjectVar
from dcim.models import Device

class BulkStatusUpdate(Script):
    class Meta:
        name = "Bulk Device Status Update"
        description = "複数デバイスのステータスを一括変更"

    devices = MultiObjectVar(
        model=Device,
```

```
        description="対象デバイスを選択"
    )

    def run(self, data, commit):
        for device in data['devices']:
            device.status = 'active'
            device.save()
            self.log_success(f"{device.name} を Active に変更しました")
```

8.7 設定コンテキスト（Config Contexts）

デバイスや仮想マシンに構造化データ（JSON / YAML）を付与できます。 Ansible の group_vars や host_vars のような使い方が可能です。

Customization > Config Contexts から作成します。

8.7.1 活用例

NTP サーバーの設定を地域ごとに付与：

```
{
  "ntp": {
    "servers": [
      "ntp1.example.com",
      "ntp2.example.com"
    ],
    "timezone": "Asia/Tokyo"
  }
}
```

適用条件：Regions = Japan

デバイス詳細画面の「Config Context」タブで、適用されたすべてのコンテキストがマージされた結果を確認できます。

9 API と自動化

NetBox は REST API と GraphQL API を提供しており、外部システムとの連携や自動化が容易です。

9.1 API トークンの発行

9.1.1 Web UI からの発行

1. 右上のユーザーアイコン > **API Tokens** をクリック
2. 「+ Add a Token」をクリック
3. トークンの説明を入力して「Save」

9.1.2 API トークンの管理

設定	説明
Key	トークン文字列（40 文字）
Write Enabled	書き込み権限の有無
Expires	有効期限（任意）
Allowed IPs	利用を許可する IP アドレス（任意）
Description	説明

9.2 REST API の基本

9.2.1 API ドキュメント

NetBox は Swagger UI を内蔵しています。 `http://<NetBox URL>/api/docs/` でインタラクティブな API ドキュメントを確認できます。

9.2.2 基本的なリクエスト

```
# デバイス一覧の取得
curl -H "Authorization: Token <your-token>" \
     -H "Content-Type: application/json" \
     http://netbox.example.com/api/dcim/devices/

# 特定デバイスの取得
curl -H "Authorization: Token <your-token>" \
     http://netbox.example.com/api/dcim/devices/?name=core-sw-01

# デバイスの作成
```

```
curl -X POST \
     -H "Authorization: Token <your-token>" \
     -H "Content-Type: application/json" \
     -d '{"name": "access-sw-01", "device_type": 1, "device_role": 2, "site": 3}' \
     http://netbox.example.com/api/dcim/devices/
```

9.2.3 フィルタリング

```
# サイトでフィルタ
GET /api/dcim/devices/?site=tokyo-dc

# ステータスでフィルタ
GET /api/dcim/devices/?status=active

# 複数条件（AND）
GET /api/dcim/devices/?site=tokyo-dc&role=switch

# 文字列検索
GET /api/dcim/devices/?q=core-sw
```

9.2.4 ページネーション

API レスポンスには `count`、`next`、`previous`、`results` が含まれます。

```
# 最初の 50 件（デフォルト）
GET /api/dcim/devices/

# 件数を変更
GET /api/dcim/devices/?limit=100&offset=0

# 全件取得（limit=0）
GET /api/dcim/devices/?limit=0
```

9.3 Python からの API 利用

9.3.1 requests ライブラリを使う場合

```
import requests

NETBOX_URL = "http://netbox.example.com"
TOKEN = "your-api-token"

headers = {
    "Authorization": f"Token {TOKEN}",
    "Content-Type": "application/json",
```

```
}

# デバイス一覧の取得
response = requests.get(
    f"{NETBOX_URL}/api/dcim/devices/",
    headers=headers,
    params={"site": "tokyo-dc", "status": "active"}
)
devices = response.json()["results"]

for device in devices:
    ip = device['primary_ip4']['address'] if device['primary_ip4'] else 'N/A'
    print(f"{device['name']}: {ip}")
```

9.3.2 pynetbox ライブラリ（推奨）

公式 Python クライアントライブラリ `pynetbox` を使うと、より簡潔に書けます。

```
pip install pynetbox
```

```
import pynetbox

nb = pynetbox.api(
    url="http://netbox.example.com",
    token="your-api-token"
)

# デバイス一覧
devices = nb.dcim.devices.filter(site="tokyo-dc", status="active")
for device in devices:
    ip = device.primary_ip4.address if device.primary_ip4 else 'N/A'
    print(device.name, ip)

# デバイスの作成
device_type = nb.dcim.device_types.get(slug="catalyst-9300-48p")
device_role = nb.dcim.device_roles.get(slug="access-switch")
site = nb.dcim.sites.get(slug="tokyo-dc")

new_device = nb.dcim.devices.create(
    name="access-sw-02",
    device_type=device_type.id,
    device_role=device_role.id,
    site=site.id,
    status="active",
)
print(f"Created: {new_device}")

# IP アドレスの割り当て
```

```
ip = nb.ipam.ip_addresses.create(
    address="10.0.1.10/24",
    status="active",
    assigned_object_type="dcim.interface",
    assigned_object_id=new_device.interfaces()[0].id,
)
```

9.4 GraphQL API

REST API に加え、GraphQL API も利用できます。

```
POST /graphql/
```

```
query {
  device_list(site: "tokyo-dc", status: "active") {
    name
    primary_ip4 {
      address
    }
    interfaces {
      name
      enabled
      ip_addresses {
        address
      }
    }
  }
}
```

GraphQL はネストしたデータを 1 回のリクエストで取得でき、 過剰なデータ取得を防げます。

9.5 Ansible との連携

9.5.1 netbox.netbox コレクション

Ansible Galaxy から NetBox 公式コレクションをインストールします。

```
ansible-galaxy collection install netbox.netbox
```

9.5.2 Dynamic Inventory（動的インベントリ）

```
# inventory/netbox.yml
plugin: netbox.netbox.nb_inventory
api_endpoint: http://netbox.example.com
token: your-api-token
```

```yaml
validate_certs: false
group_by:
  - site
  - device_roles
  - platforms
device_query_filters:
  - status: active
compose:
  ansible_host: primary_ip4.address | ipaddr('address')
```

```
ansible-inventory -i inventory/netbox.yml --list
```

9.5.3 Ansible モジュールの使用例

```yaml
- name: NetBox にデバイスを登録
  netbox.netbox.netbox_device:
    netbox_url: http://netbox.example.com
    netbox_token: "{{ netbox_token }}"
    data:
      name: "{{ inventory_hostname }}"
      device_type: "{{ device_type }}"
      device_role: "{{ device_role }}"
      site: "{{ site }}"
      status: active
    state: present
```

9.6 Terraform との連携

```
terraform {
  required_providers {
    netbox = {
      source  = "e-breuninger/netbox"
      version = "~> 3.0"
    }
  }
}

provider "netbox" {
  server_url = "http://netbox.example.com"
  api_token  = var.netbox_token
}

data "netbox_prefix" "management" {
  prefix = "10.0.2.0/24"
}
```

```
resource "netbox_available_ip_address" "vm_ip" {
  prefix_id   = data.netbox_prefix.management.id
  status      = "active"
  dns_name    = "web-server-03.example.com"
  description = "Terraform で割り当て"
}
```

9.7 NAPALM 連携

NetBox は NAPALM と連携して、実機から設定情報を取得できます。

デバイスの Platform に NAPALM ドライバーを設定します：

- `ios`（Cisco IOS）
- `iosxr`（Cisco IOS-XR）
- `eos`（Arista EOS）
- `junos`（Juniper Junos）
- `nxos`（Cisco NX-OS）

デバイス詳細画面の「NAPALM」タブから以下の情報を取得できます：

- Facts（基本情報）
- Interfaces（インターフェース状態）
- ARP テーブル
- LLDP ネイバー

10 運用管理

10.1 バックアップとリストア

10.1.1 データベースのバックアップ

定期的なバックアップを設定します。

```
#!/bin/bash
# backup_netbox.sh

BACKUP_DIR="/opt/backups/netbox"
DATE=$(date +%Y%m%d_%H%M%S)

mkdir -p "$BACKUP_DIR"

# データベースのダンプ
docker compose -f /opt/netbox-docker/docker-compose.yml \
  exec -T postgres pg_dump -U netbox netbox | \
  gzip > "$BACKUP_DIR/netbox_db_$DATE.sql.gz"

# メディアファイル（画像・添付ファイル）のバックアップ
docker compose -f /opt/netbox-docker/docker-compose.yml \
  exec netbox tar -czf - /opt/netbox/netbox/media/ | \
  cat > "$BACKUP_DIR/netbox_media_$DATE.tar.gz"

# 古いバックアップを削除（30 日以上前）
find "$BACKUP_DIR" -mtime +30 -delete

echo "Backup completed: $DATE"
```

cron に登録：

```
# 毎日 2:00 にバックアップ
0 2 * * * /opt/scripts/backup_netbox.sh >> /var/log/netbox_backup.log 2>&1
```

10.1.2 リストア

```
# データベースのリストア
gunzip < netbox_db_20240101_020000.sql.gz | \
  docker compose exec -T postgres psql -U netbox netbox

# メディアファイルのリストア
cat netbox_media_20240101_020000.tar.gz | \
  docker compose exec -T netbox tar -xzf - -C /
```

10.2 アップグレード

10.2.1 アップグレード前の確認

1. リリースノートを確認（破壊的変更がないか）
2. 現在のバージョンを確認
3. バックアップを取得
4. ステージング環境でテスト

```
# 現在のバージョン確認
docker compose exec netbox python /opt/netbox/netbox/manage.py version
```

10.2.2 アップグレード手順（netbox-docker）

```
cd /opt/netbox-docker

# リリースブランチを更新
git pull

# 新しいイメージを取得
docker compose pull

# 再起動
docker compose up -d

# マイグレーションの実行（自動で行われるが確認）
docker compose exec netbox python /opt/netbox/netbox/manage.py migrate

# ログ確認
docker compose logs -f netbox
```

10.3 パフォーマンスチューニング

10.3.1 データベースの最適化

```
# VACUUM（不要データの解放）
docker compose exec postgres psql -U netbox -c "VACUUM ANALYZE;"

# インデックス統計の更新
docker compose exec postgres psql -U netbox -c "ANALYZE;"
```

10.3.2 Redis キャッシュ

NetBox は Redis をキャッシュとジョブキューに使用しています。

```
# Redis の状態確認
docker compose exec redis redis-cli info | grep -E "used_memory|connected_clients"
```

10.3.3 Workers の設定

バックグラウンドジョブ（Webhook、スクリプト実行など）は `rqworker` が処理します。

```
# docker-compose.override.yml
services:
  netbox-worker:
    deploy:
      replicas: 2  # ワーカー数を増加
```

10.4 監視

10.4.1 ヘルスチェックエンドポイント

```
GET /api/status/
```

```
{
  "django-version": "4.2.x",
  "installed-apps": {...},
  "netbox-version": "4.x.x",
  "plugins": [],
  "python-version": "3.11.x"
}
```

10.4.2 アクセスログ

```
# Nginx アクセスログ（コンテナ内）
docker compose logs nginx

# NetBox アプリログ
docker compose logs netbox | grep ERROR
```

10.4.3 Prometheus / Grafana との連携

`django-prometheus` を導入することで、メトリクスを Prometheus で収集し Grafana で可視化できます。

10.5 プラグイン

NetBox はプラグインによる機能拡張をサポートしています。

10.5.1 主要なプラグイン

プラグイン	機能
netbox-bgp	BGP ピア・セッション管理
netbox-topology-views	ネットワークトポロジー図の生成
netbox-floorplan-plugin	フロアプラン・ラック配置図
netbox-secrets	機密情報（パスワード等）の暗号化管理
netbox-dns	DNS ゾーン・レコード管理
netbox-acls	ACL 管理

10.5.2 プラグインのインストール（Docker）

`configuration/plugins.py` に設定を追加します：

```
PLUGINS = ['netbox_bgp']

PLUGINS_CONFIG = {
    'netbox_bgp': {
        'device_ext_page': 'right',
    }
}
```

`local_requirements.txt` にパッケージを追加します：

```
netbox-bgp
```

コンテナを再ビルドします：

```
docker compose build
docker compose up -d
```

10.6 セキュリティのベストプラクティス

10.6.1 本番環境向けの設定

```
# configuration.py

# HTTPS 強制
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True

# デバッグ無効
```

```
DEBUG = False

# 許可するホストを明示
ALLOWED_HOSTS = ['netbox.example.com']

# ログイン必須
LOGIN_REQUIRED = True

# セッションタイムアウト（30分）
LOGIN_TIMEOUT = 1800

# SECRET_KEY は環境変数から取得
import os
SECRET_KEY = os.environ.get('SECRET_KEY')
```

10.6.2 LDAP / SSO 連携

Active Directory や LDAP との認証連携が可能です。

```
pip install django-auth-ldap
```

```
# ldap_config.py
import ldap
from django_auth_ldap.config import LDAPSearch, GroupOfNamesType

AUTH_LDAP_SERVER_URI = "ldap://ldap.example.com"
AUTH_LDAP_BIND_DN = "cn=netbox,ou=service,dc=example,dc=com"
AUTH_LDAP_BIND_PASSWORD = "password"

AUTH_LDAP_USER_SEARCH = LDAPSearch(
    "ou=users,dc=example,dc=com",
    ldap.SCOPE_SUBTREE,
    "(uid=%(user)s)"
)
```

10.6.3 定期的なセキュリティタスク

- API トークンの定期的なローテーション
- 不要なユーザー・トークンの削除
- アクセスログの定期的なレビュー
- NetBox および依存パッケージの定期的なアップデート
- 脆弱性情報の購読（GitHub Security Advisories）

10.7 よくあるトラブルと対処法

10.7.1 DB 接続エラー

```
# PostgreSQL コンテナの状態確認
docker compose ps postgres
docker compose logs postgres
```

10.7.2 マイグレーションの失敗

```
# マイグレーション状態の確認
docker compose exec netbox python manage.py showmigrations | grep "\[ \]"

# マイグレーションの再実行
docker compose exec netbox python manage.py migrate
```

10.7.3 パフォーマンス低下

```
# 遅いクエリの特定
docker compose exec postgres psql -U netbox -c \
  "SELECT query, mean_exec_time, calls FROM pg_stat_statements ORDER BY mean_exec_time
DESC LIMIT 10;"
```